

Technical Report : Deep Metric Learning for Trip Advisor Reviews Based Recommender

Authors :

BACHABI Rochdy
SAINT ANDRE Jeffrey

Information Retrieval & NLP
Mars 2026

Table of Contents

1	Introduction	2
2	Model Architecture : Attention-Based Aggregator	2
2.1	Variant : Mean Pooling Approach	3
3	Dataset and Sampling Strategy	3
4	Optimization : Batch-All Triplet Loss	3
5	Inference and Scalability	3
6	Evaluation Metrics	3
6.1	Recall@K and Ranking Error	3
6.2	Business Evaluation : Weighted Coherence Score (WCS)	4
7	Results	4
7.1	Training and Convergence	4
7.2	Quantitative Performance Comparison	4
7.3	Qualitative Analysis and Inference	5
8	Conclusion	6

1 Introduction

This report describes a neural architecture designed to recommend tourist locations by mapping sets of reviews into a shared latent space. By leveraging Deep Metric Learning, the system generates a unique "place signature" from text embeddings. This approach allows for high-precision retrieval and seamless scalability to new locations without model representation.

To benchmark our neural approach, we first implemented a BM25 (Best Matching 25) baseline. This lexical retrieval model uses term frequency and inverse document frequency to rank locations. It provides a control group to measure the semantic gain achieved by our Attention based model.

2 Model Architecture : Attention-Based Aggregator

To transform a variable number of input reviews (k) into a fixed-dimensional vector, we primarily focused on an Attention Aggregator/Encoder. The motivation was that, unlike simple averaging, this mechanism identifies which reviews are most "place-defining".

- **Permutation Invariance** : The model treats the input as a set rather than a sequence, ensuring that the order of reviews does not change the resulting signature.
- **Mechanism** : Each review embedding e_i is passed through a linear layer to compute an **importance score**. These scores are normalized via a Softmax function to produce the final weighted sum :

$$V_{place} = \sum_{i=1}^k \alpha_i e_i \quad (1)$$

where

$$\alpha = \text{Softmax}(\text{Linear}(e)) \quad (2)$$

FIGURE 1 – Model Architecture

```
model = AttentionAggregatorEncoder(384, device)

criterion = BatchAllTripletLoss(margin=0.3).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
checkpoint_manager = CheckpointManager("checkpoints")

num_epochs = 5
max_step_per_epoch = 256

start = time.perf_counter()
history = train(
    model, train_dataloader, val_dataset,
    optimizer, criterion,
    num_epochs, max_step_per_epoch,
    checkpoint_manager
)
end = time.perf_counter()

Epoch 1/5 - Loss: 0.0727 - Recall@5: 0.9828
New best Recall@k: 0.9828
Epoch 2/5 - Loss: 0.0603 - Recall@5: 1.0000
New best Recall@k: 1.0000
Epoch 3/5 - Loss: 0.0596 - Recall@5: 1.0000
Epoch 4/5 - Loss: 0.0606 - Recall@5: 0.9914
Epoch 5/5 - Loss: 0.0493 - Recall@5: 1.0000
```

As shown in the code snippet, the attention layer computes a scalar score for each review embedding. By applying a Softmax function across the review dimension, the model ensures that the total weight sum equals 1, effectively performing a context-aware weighted average that highlights the most descriptive reviews.

FIGURE 2 – Attention and weighting mechanism

```
# Importance scores
scores = self.attention_weights(x) # (batch_size, k, 1)
weights = F.softmax(scores, dim=1) # Normalisation over k reviews
```

2.1 Variant : Mean Pooling Approach

In parallel with the attention aggregator, we explored a simplified architecture based on Mean Pooling. In this configuration, the "Place Signature" is obtained by simply calculating the arithmetic mean of the review vectors. Although this method is fast, it treats each comment with the same importance. As a result, the semantics of highly descriptive reviews are often diluted by generic comments (like "Very good" or "Ok"), affecting the overall recommendation accuracy compared to the attention-based filtering.

3 Dataset and Sampling Strategy

The training process relies on a specialized data pipeline to simulate the recommendation task :

- **Fixed-Iteration Sampling** : For each training step, the dataset draws k random embeddings from the pool of reviews. This serves as a stochastic data augmentation, exposing the model to different "perspectives" of the same place.
- **Triples Batch Sampler** : We use a custom sampler ensuring each batch contains P distinct locations, with M instances per location. This is critical for effective negative mining.

4 Optimization : Batch-All Triplet Loss

The model is trained using a Triplet Loss function with online mining. The loss pushes the distance between an Anchor and a Positive (same place) to be smaller than the distance between the Anchor and a Negative (different place) by at least a specified margin.

$$L = \max(d(A, P) - d(A, N) + \text{margin}, 0) \quad (3)$$

The Batch-All Triplet Loss ensures that the model learns a discriminative manifold, forcing embeddings of the same location to collapse into a tight cluster, which confirms our hypothesis that similar experiences share similar linguistic signatures.

5 Inference and Scalability

- **Zero-Shot Expansion** : New locations are added by calculating their signature without requiring retraining.
- **Chunked Aggregation** : To handle landmarks with thousands of reviews, the engine processes reviews in manageable blocks to form a stable "Golden Embedding".
- **Fast Retrieval** : Using Approximate Nearest Neighbor (ANN) search allows sub-millisecond retrieval.

6 Evaluation Metrics

6.1 Recall@K and Ranking Error

We use Recall@K to measure how often the ground-truth appears in the top results. We also evaluate the system using **Ranking Error (RE)**. Level 1 measures the rank of the first result matching the query's primary category. Level 2 counts a match if there is any overlap in the multi-category list.

6.2 Business Evaluation : Weighted Coherence Score (WCS)

We implement a WCS across four dimensions (Type, Category, Rating, Price) to evaluate the "quality of failure" and the serendipity of the engine.

TABLE 1 – Business Evaluation Criteria

Criteria	Weight	Method
Type	35%	Exact match
Category	25%	Exact match
Rating	20%	Linear distance
Price	20%	Price range difference

7 Results

7.1 Training and Convergence

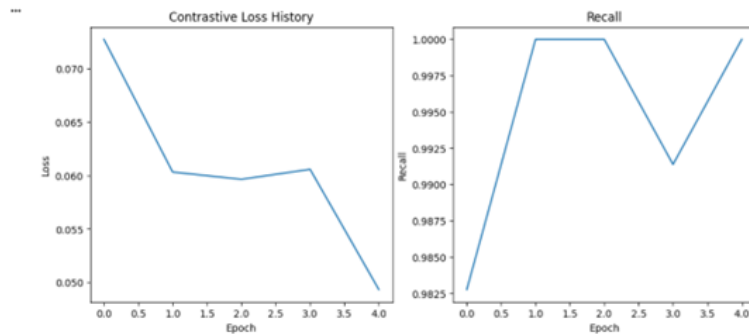


FIGURE 3 – Training Metrics : Triplet Loss and Recall@5 evolution.

The training logs demonstrate a very rapid convergence. The model reaches a perfect Recall@5 on the validation set almost immediately, while the Triplet Loss continues to decrease. This indicates that the chosen margin and the Batch-All mining strategy are highly effective at separating different location signatures.

7.2 Quantitative Performance Comparison

TABLE 2 – Comparative performance metrics (Averages)

Metric	BM25	Mean Pooling	Attention Aggregator
Ranking Error (Level 1)	5.51	4.35	4.88
Ranking Error (Level 2)	1.83	1.33	1.48
Relevance Score	0.85	0.88	0.902

We observe that if the Mean Pooling gets the best rank error (RE), the Aggregator Attention gets the best Relevance Score (0.902). This indicates that the attention mechanism is better able to extract the semantic features that correspond to the actual categories of places. It validates our initial approach.

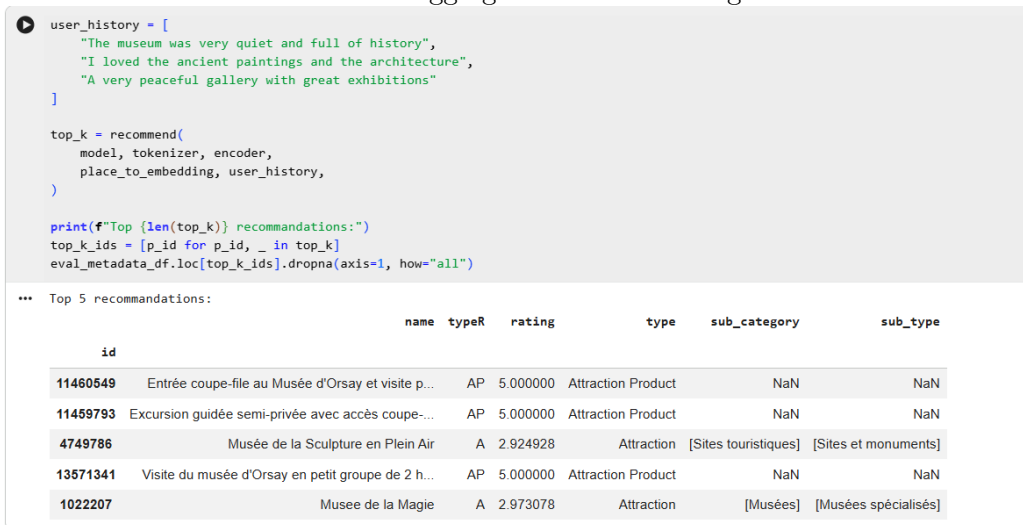
7.3 Qualitative Analysis and Inference

FIGURE 4 – Qualitative Inference using a Single Review as Query



This test validates the model's robustness when dealing with sparse data. As shown in this figure, even with a single input review, the system is capable of projecting the query into the correct semantic neighborhood. This demonstrates that the Sentence Transformer backbone effectively captures place-defining features even without the benefit of multiple-review aggregation.

FIGURE 5 – Multi-Review Aggregation for Stable Signature Generation.



We observe the inference process using a set of k reviews. By aggregating multiple perspectives, the Attention mechanism filters out individual reviewer bias. This creates a more stable embedding, which represents the consensus identity of the location rather than a single subjective experience.

FIGURE 6 – Top-5 Recommendations for the "Tournée de photoshoot à Paris" Case Study.

```
*** Top 5 recommandations for:
name      Tournée de photoshoot à Paris avec un photogra...
typeR      AP
rating     5.0
type       Attraction Product
Name: 14943088, dtype: object
```

	id	name	typeR	rating	type
	15304777	Séance de photographie privée de vacances avec...	AP	5.0	Attraction Product
	15154101	TOUR EIFFEL – Tour photographique d'une durée ...	AP	5.0	Attraction Product
	13346619	Séance photo à Paris pour les familles et les ...	AP	5.0	Attraction Product
	12471503	Visite privée: Visite personnelle de photograp...	AP	5.0	Attraction Product
	11473660	Atelier de perfectionnement	AP	5.0	Attraction Product

It illustrates the final result of the recommendation. For a query based on the photoshoot in Paris, the system was able to identify the different places and comments that are closest for a photo session. This confirms that the model perfectly understands semantic similarity by providing the closest suggestions according to the query.

8 Conclusion

The comparison between our three models highlights the importance of semantic representation. The shift from a lexical method (BM25) to neural methods significantly reduces classification error. While the Mean Pooling model is efficient, the Attention Aggregator remains the most robust approach to maximize the overall relevance of recommendations (Relevance Score), proving the value of weighting reviews based on their informative content.